

PRÁCTICA B6 · BLOQUE III – TIEMPO REAL

Streaming, filtrado robusto y clasificador de actividad

DURACIÓN	2 h	MODALIDAD	Laboratorio · equipos de dos
ENTORNO	Python · SciPy · scikit-learn	DATOS	CSV de B4 y B5 · RSSI simulado
APORTA AL PROYECTO	Filtrado del RSSI en vivo	HITO	Prepara H2

01 Objetivos

Al finalizar esta práctica el alumno habrá adquirido las siguientes competencias:

Filtrado en streaming con estado (zi): Aplicar filtros FIR/IIR a datos que llegan por paquetes sin generar glitches, usando `lfilter_zi` y la clase `RealTimeFilter` (Tema 9, Slide 40-41).

DC Blocker adaptativo: Implementar el filtro IIR $y[n] = x[n] - x[n-1] + 0.95 \cdot y[n-1]$ y demostrar su superioridad sobre restar la media cuando el sensor cambia de orientación (Tema 9, Slide 34).

Filtrado robusto de RSSI: Combinar filtro de Mediana (no lineal, inmune a outliers) con IIR suave, y verificar el diseño con el diagrama de Bode (Tema 9, Slides 42-43).

Clasificación espectral: Extraer features del espectrograma (frecuencia dominante, energía, entropía, centroide) y entrenar un clasificador KNN para distinguir andando/corriendo/mixto (Tema 8, Slides 19-21).

02 Competencias y resultados de aprendizaje

Esta práctica desarrolla los siguientes resultados de aprendizaje de la memoria verificada del título:

- **RA4** · Aprender a realizar operaciones sobre señales digitales y a obtener información de estas.
- **RA6** · Ser capaz de implementar algoritmos de procesamiento sobre señales adquiridas previamente, evaluando su comportamiento frente a datos reales. Aquí además en tiempo real, que es donde aparecen los problemas que el procesamiento por lotes esconde.
- **RA7** · Extraer características de una señal y utilizarlas para clasificar. El clasificador de actividad es el primer paso hacia la estimación de posición del proyecto.

Se trabaja la CG9, capacidad de análisis crítico: el bloque A obliga a implementar primero la versión incorrecta y a medir en qué se nota.

03 Material y datos

Software

PC con Python 3.10+ (Windows) y PyCharm Professional (licencia UVigo).

Librerías (las mismas que P4-P5 más scikit-learn):

```
pip install numpy pandas matplotlib scipy scikit-learn
```

Ficheros necesarios

Los 3 CSV de la Práctica 5, renombrados como:

Fichero	Contenido	Duración aprox.
andando.csv	Caminando a ritmo normal	30-60 s
corriendo.csv	Corriendo a ritmo sostenido	30-60 s
andando_corriendo.csv	Secuencia mixta: andar correr andar	60-90 s

TipSi tus CSV tienen columnas `ts_ms`, `ax`, `ay`, `az`, `gx`, `gy`, `gz`, están listos. La frecuencia de muestreo real se calcula como $f_s = 1000 / \text{mediana}(\text{diff}(\text{ts_ms}))$.

04 Fundamento teórico

Introducción y Contexto

En las Prácticas 4 y 5 construiste un pipeline de procesamiento offline completo: cargabas un CSV, filtrabas con Butterworth+filtfilt, detectabas pasos y reconstruías la trayectoria. Todo ello con la señal completa disponible de antemano en memoria. Ese enfoque es perfecto para análisis forense de datos, pero en un sistema IoT real los datos no llegan de golpe: llegan gota a gota, en paquetes MQTT de 5-10 muestras cada 100 ms.

Esta práctica cierra el círculo entre la teoría de los Temas 8-9 y la implementación práctica. El objetivo es triple:

Bloque A: Simular streaming MQTT y resolver el problema crítico del estado interno (zi) al filtrar por paquetes.

Bloque B: Implementar el DC Blocker adaptativo y compararlo con el método ingenuo de restar la media global.

Bloque C: Aplicar la combinación profesional Mediana + IIR al RSSI WiFi y verificar con el diagrama de Bode.

Bloque D: Construir un clasificador de actividad basado en características espectrales extraídas del espectrograma, usando los 3 CSV de la P5.

Aviso Esta práctica reutiliza los 3 ficheros CSV que grabaste en la P5 (andando.csv, corriendo.csv, andando_corriendo.csv). Si no los tienes, usa los del compañero o los de ejemplo del profesor.

Por qué un filtro necesita recordar su estado

Cuando un filtro digital procesa datos, mantiene un estado interno: los valores de las muestras anteriores (para FIR) y de las salidas anteriores (para IIR). Si se reinicia ese estado a cero en cada paquete, el filtro arranca desde cero cada vez, generando un transitorio de arranque que aparece como un glitch visible en la señal de salida.

La función `lfilter_zi(b, a)` calcula las condiciones iniciales óptimas del filtro. Al pasar `zi` como parámetro a `lfilter`, el filtro continúa exactamente donde lo dejó, como si nunca se hubiera interrumpido.

El DC blocker como paso alto de un polo

El DC Blocker es un filtro paso-alto de primer orden definido por la ecuación en diferencias:

$$y[n] = x[n] - x[n-1] + R \cdot y[n-1] \quad \text{donde } R = 0.95$$

El parámetro `R` controla la frecuencia de corte inferior. Con `R=0.95` y `fs=50` Hz, el filtro bloquea frecuencias por debajo de ~0.4 Hz, eliminando la gravedad pero preservando el movimiento humano (>1 Hz). La clave es que se adapta automáticamente: si el usuario rota el teléfono y la DC cambia de 9.8 a 7.0 m/s², el filtro se reajusta en menos de 1 segundo.

Por qué la mediana y no la media en el RSSI

El RSSI WiFi sufre de multipath y desvanecimiento: puntualmente puede saltar de -55 dBm a -90 dBm en una sola muestra. Un filtro IIR Butterworth, al ser lineal, reacciona proporcionalmente a ese pico: la salida se hunde. El filtro de Mediana, al tomar el valor central de las últimas `k` muestras, ignora completamente los outliers.

La receta profesional para localización WiFi es: primero mediana (quita outliers), luego IIR suave (elimina ruido residual).

Características espectrales y clasificación

En lugar de alimentar la IA con cientos de muestras crudas, extraemos 4 valores resumen del espectro de cada ventana STFT:

1. Frecuencia Dominante (`f_dom`): El pico máximo del espectro. Correlaciona con la cadencia de los pasos.
2. Energía Espectral Total: Suma de $|X[k]|^2$. Indica la intensidad del esfuerzo físico.
3. Entropía Espectral: Mide cuán puntiagudo o plano es el espectro. Baja = señal rítmica. Alta = ruido caótico.

4. Centroide Espectral: Centro de masa del espectro. Indica si la vibración es grave (caminar) o aguda (correr).

Con solo estos 4 números por ventana, un clasificador KNN con $k=5$ puede separar las actividades con alta precisión.

05 Procedimiento

Bloque A · Streaming y el problema del z_i

Demostrar que aplicar `lfilter` a cada paquete MQTT por separado genera glitches periódicos, e implementar la solución correcta: la clase `RealTimeFilter` que preserva el estado interno (z_i) entre llamadas. Este es, según el Tema 9, el concepto más importante del procesamiento en streaming.

Bloque A · Tareas

Tarea A1 — Carga y preparación

Carga `andando.csv`. Calcula la magnitud $|a| = \sqrt{a_x^2 + a_y^2 + a_z^2}$. Calcula la f_s real. Diseña un filtro FIR paso-bajo con `firwin(31, 5.0, fs=fs)`.

Tarea A2 — Filtrado INCORRECTO (sin z_i)

Simula que los datos llegan en chunks de `CHUNK=10` muestras. Aplica `lfilter(b, a, chunk)` a cada chunk por separado, sin pasar z_i . Concatena los resultados.

```
filtered_bad = np.zeros(len(mag))
for i in range(0, len(mag), CHUNK):
    chunk = mag[i:i+CHUNK]
    y = lfilter(b_fir, a_fir, chunk) # ERROR: sin z_i
    filtered_bad[i:i+len(y)] = y
```

Tarea A3 — Filtrado CORRECTO (con z_i)

Implementa la clase `RealTimeFilter` y repite el filtrado:

```

from scipy.signal import lfilter, lfilter_zi

class RealTimeFilter:
    def __init__(self, b, a):
        self.b, self.a = b, a
        self.zi = lfilter_zi(b, a) * 0.0

    def process_chunk(self, chunk):
        y, self.zi = lfilter(self.b, self.a, chunk, zi=self.zi)
        return y

stream_filter = RealTimeFilter(b_fir, a_fir)
filtered_good = np.zeros(len(mag))
for i in range(0, len(mag), CHUNK):
    chunk = mag[i:i+CHUNK]
    filtered_good[i:i+len(y)] = stream_filter.process_chunk(chunk)

```

Tarea A4 — Referencia offline y comparación

Aplica `lfilter(b, a, mag)` a toda la señal de una vez (referencia offline). Genera una figura de 3 subplots: (1) sin `zi`, (2) con `zi`, (3) superposición de `zi` vs offline. Calcula el error máximo entre ambas.

Bloque A · Experimentos

Exp. A1 — Efecto del tamaño de chunk

Repite el filtrado incorrecto (sin `zi`) con `CHUNK = 5, 10, 20, 50`. Anota en qué tamaño el glitch se hace visualmente intolerable y en cuál es apenas perceptible.

CHUNK	Glitch visible	Error máximo vs offline
5		
10		
20		
50		

Exp. A2 — FIR vs IIR en streaming

Repite el experimento con un filtro IIR Butterworth orden 4, `fc=5 Hz` en lugar del FIR. ¿El glitch sin `zi` es más o menos grave que con FIR? ¿Por qué?

TipEl IIR tiene feedback (coeficientes `a`), por lo que su estado interno es más crítico. El transitorio de arranque suele ser peor que en un FIR.

Bloque B · DC blocker adaptativo

Implementar el filtro DC Blocker IIR que elimina la componente continua (gravedad) del acelerómetro de forma adaptativa, y demostrar que es superior a restar la media global cuando la orientación del sensor cambia durante la captura.

Bloque B · Tareas

Tarea B1 — Simulación de rotación

Genera una señal sintética de 10 segundos a 50 Hz donde: la gravedad es 9.8 m/s^2 durante los primeros 4 segundos; en $t=4\text{s}$ el sensor rota y la DC cae exponencialmente a 7.0 m/s^2 ; hay movimiento de caminar superpuesto (2 Hz, amplitud 1.5 m/s^2).

Tarea B2 — Método ingenuo: signal - mean

Resta la media global de toda la señal. Observa que después de la rotación aparece un offset residual permanente.

Tarea B3 — DC Blocker IIR

Implementa el DC Blocker con un bucle for:

```
R = 0.95
dc_blocked = np.zeros(N)
x_prev, y_prev = signal[0], 0.0
for i in range(N):
    dc_blocked[i] = signal[i] - x_prev + R * y_prev
    x_prev = signal[i]
    y_prev = dc_blocked[i]
```

Tarea B4 — Aplicación a datos reales

Aplica el DC Blocker a la señal `|a|` de `andando.csv` y compáralo con el método de restar la media. Genera una figura con 3 subplots: (1) señal cruda, (2) signal-mean, (3) DC Blocker.

Bloque B · Experimentos

Exp. B1 — Efecto de R

Prueba $R = 0.8, 0.9, 0.95, 0.99$ sobre la señal sintética con rotación. Anota cuánto tarda cada uno en recuperarse tras la rotación:

R	Tiempo de recuperación (s)	Atenuación del paso (2 Hz)
0.80		
0.90		
0.95		
0.99		

Exp. B2 — DC Blocker en streaming

Combina el DC Blocker con la clase `RealTimeFilter` del Bloque A. El DC Blocker, al ser IIR de primer orden, también necesita preservar su estado `zi` entre chunks. Verifica que funciona correctamente comparando

con el resultado offline.

Bloque C · Filtrado robusto de RSSI WiFi

Demostrar que los filtros lineales (FIR/IIR) se ven arrastrados por outliers impulsivos del RSSI WiFi, implementar la combinación profesional Mediana + IIR, y verificar el diseño del filtro con el diagrama de Bode.

Bloque C · Tareas

Tarea C₁ — Simulación de RSSI con outliers

Genera 200 muestras de RSSI a $f_s=10$ Hz: tendencia lenta senoidal (-55 ± 8 dBm a 0.03 Hz) + ruido gaussiano ($\sigma=3$ dB) + 5 outliers impulsivos de -35 dB inyectados en posiciones aleatorias.

Tarea C₂ — Filtro IIR Butterworth

Diseña un filtro IIR Butterworth paso-bajo de orden 2, frecuencia de corte 0.5 Hz, y aplícalo con `lfilter`. Observa cómo se hunde con cada outlier.

Tarea C₃ — Filtro de Mediana

Aplica `medfilt(rssi, kernel_size=11)` de `scipy.signal`. Observa cómo ignora los outliers.

Tarea C₄ — Combo profesional + Bode

Aplica primero la mediana y luego el IIR sobre el resultado. Genera una figura con: (1) panel superior con las 4 señales superpuestas (crudo, IIR, mediana, combo); (2) panel inferior con el diagrama de Bode del IIR usando `scipy.signal.freqz`.

```
# Diagrama de Bode
from scipy.signal import freqz
w, h = freqz(b_iir, a_iir, worN=1024, fs=fs)
plt.plot(w, 20*np.log10(np.abs(h)))
plt.axvline(fc, color='r', linestyle='--', label=f'fc={fc} Hz')
plt.axhline(-3, color='gray', linestyle=':', label='-3 dB')
```

Bloque C · Experimentos

Exp. C₁ — Tamaño de kernel de mediana

Prueba `kernel_size = 3, 7, 11, 21`. Anota cuál elimina mejor los outliers sin distorsionar excesivamente la tendencia. Recuerda que `kernel_size` debe ser impar.

Exp. C₂ — `filtfilt` vs `lfilter`

Aplica el IIR con `filtfilt` (offline, sin retardo) y con `lfilter` (causal, con retardo). Superpone ambas en la misma gráfica. Mide el retardo de grupo en muestras. Pregunta: ¿cuál usarías en el ESP32 en tiempo real y cuál en el backend Python?

Bloque D · Clasificador de actividad espectral

Extraer características espectrales del espectrograma de cada CSV, entrenar un clasificador KNN (K-Nearest Neighbors) y evaluar su capacidad para distinguir automáticamente entre andando, corriendo y mixto. Este bloque conecta los conceptos del Tema 8 (espectrograma como entrada a ML) con una implementación práctica de TinyML.

Bloque D · Tareas

Tarea D1 — Extracción de features por ventana

Para cada CSV, calcula $|a|$, aplica DC Blocker y divide la señal en ventanas de 128 muestras con 50% de overlap. Para cada ventana, calcula las 4 features:

```
def extraer_features(ventana, fs):
    N = len(ventana)
    freqs = np.fft.rfftfreq(N, 1/fs)
    mag = 2/N * np.abs(np.fft.rfft(ventana * np.hanning(N)))
    mag = mag[1:] # Excluir DC
    f = freqs[1:]

    f_dom = f[np.argmax(mag)]
    energia = np.sum(mag**2)
    m_norm = mag / (np.sum(mag) + 1e-10)
    entropia = -np.sum(m_norm * np.log2(m_norm + 1e-10))
    centroide = np.sum(f * mag) / (np.sum(mag) + 1e-10)

    return [f_dom, energia, entropia, centroide]
```

Tarea D2 — Construcción del dataset

Etiqueta cada ventana con su actividad (0=andando, 1=corriendo). Para andando_corriendo.csv, segmenta manualmente observando el espectrograma de la P5. Construye una matriz X ($n_{\text{ventanas}} \times 4$) y un vector y (n_{ventanas}).

Tarea D3 — Entrenamiento y evaluación KNN

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix, classification_report

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y)

knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)
y_pred = knn.predict(X_test)

print(classification_report(y_test, y_pred,
    target_names=['Andando', 'Corriendo']))
```

Tarea D4 — Visualización del espacio de features

Genera un scatter plot de `f_dom` vs energía, coloreado por clase. Superpone los bordes de decisión del KNN si es posible. Genera la matriz de confusión como heatmap.

Bloque D · Experimentos

Exp. D1 — Número de vecinos

Prueba $k = 1, 3, 5, 7, 11$ y anota la precisión (accuracy) para cada uno:

k (vecinos)	Accuracy (%)	Observaciones
1		
3		
5		
7		
11		

Exp. D2 — Feature importance

Entrena 4 clasificadores KNN usando solo 1 feature cada vez (solo `f_dom`, solo energía, solo entropía, solo centroide). ¿Cuál feature individual separa mejor las clases? ¿Es consistente con la teoría del Tema 5?

Exp. D3 — Tabla comparativa final (pieza central de la entrega)

Con tu mejor configuración (k óptimo, las 4 features), rellena esta tabla:

CSV	Ventanas andando	Ventanas corriendo	Accuracy (%)	F1-score andando	F1-score corriendo
andando.csv					
corriendo.csv					
andando_corriendo.csv					

06 Entregables

Qué debes subir

La entrega consiste en un único fichero Python que contenga los Bloques A, B, C y D completos. El fichero debe generar las figuras de todos los bloques al ejecutarse y debe imprimir por consola la tabla comparativa del Exp. D3 y el `classification_report` del KNN.

Nombre del fichero

Sin tildes, espacios ni mayúsculas. Formato:

Plazo y condiciones

Plazo: 7 días naturales desde la sesión de práctica (cierre automático a las 23:59). Entrega individual. El código se comparará automáticamente para detectar similitud $> 80\%$. El fichero debe ejecutarse sin errores con los 3 CSV en la misma carpeta.

Preguntas de Reflexión

Incluye la respuesta a al menos una de estas preguntas como comentario en tu fichero de entrega:

P1. En el Bloque A, el IIR genera un glitch peor que el FIR al reiniciar el zi. Explica por qué usando los conceptos de polos y retroalimentación del Tema 9.

P2. El parámetro R del DC Blocker controla el compromiso entre velocidad de adaptación y atenuación de bajas frecuencias. Si tu aplicación fuera detección de caídas (evento rápido), ¿subirías o bajarías R respecto a 0.95? Justifica.

P3. En el Bloque C, la mediana no es un filtro lineal. ¿Qué consecuencia tiene esto para el principio de superposición? ¿Por qué no se puede representar con coeficientes b y a como un FIR/IIR?

P4. El clasificador KNN del Bloque D funciona bien con 4 features. Si tuvieras que ejecutarlo en el ESP32 (TinyML), ¿cuál de las 4 features eliminarías para ahorrar cálculo y por qué?

P5. El Tema 8 propone usar CNNs sobre espectrogramas como imágenes 2D. Compara las ventajas e inconvenientes de ese enfoque frente al KNN con 4 features que has implementado aquí.

Bloque E – Localización por fingerprinting WiFi (k-NN)

Nuevo en el curso 2026-2027. Este bloque conecta la práctica con el proyecto integrador AINI: reutiliza el k-NN que ya conoces, pero ahora para estimar la POSICIÓN a partir de la señal WiFi.

E.1 · De clasificar la actividad a estimar la posición

En el Bloque D usaste k-NN sobre características espectrales para decidir QUÉ hace el usuario (andar / correr / mixto). En este bloque usas la misma idea —k-NN— pero para decidir DÓNDE está: comparas el vector de potencias WiFi observado contra un mapa de huellas (fingerprints) medido de antemano. Es el método C del proyecto y el corazón del posicionamiento por fingerprinting (RADAR, Bahl y Padmanabhan, 2000).

E.2 · Fundamento

Cada punto de referencia se describe por su HUELLA: un vector indexado por BSSID con la potencia (RSSI) media de cada red visible. Los BSSID no observados se rellenan con un valor bajo (-100 dBm). Para localizar un escaneo nuevo:

- Alinear el escaneo con la lista global de BSSID vector v .
- Calcular la distancia euclídea en el espacio RSSI entre v y cada huella.

- Tomar los $k = 3$ vecinos más próximos, con peso $1 / (d + 0,1)$.
- Asignar planta y zona por el vecino más cercano; las coordenadas por media ponderada.

Importante: la distancia se mide en dBm, no en metros. Es una distancia en el «espacio de señal»: dos puntos están «cerca» si ven las mismas redes con potencias parecidas. Por eso los BSSID ausentes se rellenan con -100 dBm (muy débil) y no con 0 (que sería una señal fortísima e inexistente).

E.3 · Ejercicio

Ficheros de partida (carpeta datos/):

- fingerprints_demo.csv — mapa de radio de ejemplo: 8 puntos en 2 plantas, 6 APs.
- test_scans_demo.csv — 3 escaneos de prueba, con su planta y zona reales.

Plantilla: code/fingerprint_knn.py. Completa los 3 TODOs:

- TODO 1 — distancia euclídea en el espacio RSSI entre el escaneo y cada huella.
- TODO 2 — los k vecinos más próximos y sus pesos $1 / (d + 0,1)$.
- TODO 3 — planta y zona del vecino más cercano; coordenadas por media ponderada.

Al completarlos, ejecuta el script: debe acertar la planta y la zona de los 3 escaneos de prueba (lo imprime por pantalla). Responde después:

- ¿Por qué se rellenan los BSSID ausentes con -100 dBm y no con 0?
- Si dos plantas comparten los mismos APs, ¿por qué conviene decidir la planta por votación de los APs conocidos y no solo por la huella completa?
- ¿Cómo cambia el resultado si reduces k a 1? ¿Y si añades APs vistos en un solo punto?

E.4 · Conexión con el proyecto AINI

Este es exactamente el método C que corre en el servidor de tu proyecto. Lo que aquí pruebas con 8 puntos de ejemplo, en el proyecto lo aplicarás sobre tu PROPIO mapa de radio (Hito 1) para localizar al usuario en tiempo real. En el Bloque 7 lo fundirás con la navegación inercial (PDR) para obtener una trayectoria suave.

E.5 · Entregable

- fingerprint_knn.py completo y funcionando, con una captura de la salida (aciertos).
- Respuestas breves a las preguntas de E.3.

07 Criterios de evaluación

Rúbrica de evaluación

Bloque	Criterio	Peso
A	RealTimeFilter funcional, figura de 3 subplots, tabla Exp. A1	25%
B	DC Blocker implementado, comparación con mean, tabla Exp. B1	20%
C	Mediana + IIR + Bode, comparación filtfilt vs lfilter	20%
D	Features espectrales, KNN, confusion matrix, tabla Exp. D3	30%
Reflexión	Respuesta razonada a al menos 1 pregunta de reflexión	5%